Chapter 2 Data and Expressions

CHAPTER OBJECTIVES

- Discuss the use of character strings, concatenation, and escape sequences.
- Explore the declaration and use of variables.
- Describe the Java primitive data types.
- Discuss the syntax and processing of expressions.
- Define the types of data conversions and the mechanisms for accomplishing them.
- Introduce the Scanner class to create interactive programs.

This chapter examines some of the foundational data types that appear in Java programs, along with the use of expressions to carry out computations. It also covers how data can be converted from one type to another, and how to accept input interactively from the user while a program is running.

2.1 Character Strings

In Chapter 1 we examined the fundamental structure of a Java program — its comments, identifiers, and use of white space — using the `Lincoln` program as our guiding example. That chapter also provided a high-level survey of object-oriented concepts such as objects, classes, and methods. Take a moment to revisit those ideas if a refresher would be helpful.

In Java, a character string is an object defined by the `String` class. Because strings are so central to virtually every programming task, Java allows developers to express them as string literals, delimited by double quotation marks, as we have already seen in earlier examples. A more thorough exploration of the `String` class and its methods appears in Chapter 3. For now, let's take a closer look at how string literals can be used.

The following are all valid string literals:

```
"The quick brown fox jumped over the lazy dog."
"2201 Birch Leaf Lane, Blacksburg, VA 24060"
"x"
""
```

A string literal may contain any valid characters, including numeric digits, punctuation marks, and other special symbols. The final example in the list above is an empty string — one that contains no characters whatsoever.

The `print` and `println` Methods

In the `Lincoln` program from Chapter 1, we called the `println` method as follows:

```
System.out.println ("Whatever you are, be a good one.");
```

This statement offers a concrete illustration of how objects are used. The `System.out` object represents an output destination — by default, the monitor screen. To be more precise, the object itself is named `out` and resides within the `System` class. We will examine that relationship more closely at the appropriate point in the text.

The `println` method is a service that the `System.out` object makes available to us. Whenever we request it, the object outputs a string of characters to the screen. We can think of this as sending the `println` message to the `System.out` object, instructing it to display the specified text.

Each piece of data we pass to a method is referred to as a parameter. In this particular case, `println` accepts a single parameter: the string of characters to be displayed.

The `System.out` object also exposes a second output service: the `print` method. The distinction between `print` and `println` is subtle but significant.

KEY CONCEPT
The `print` and `println` methods represent two services provided by the `System.out` object.

The `println` method outputs the data it receives and then advances the cursor to the beginning of the next line. The `print` method behaves identically in terms of output, but does not move to the next line upon completion — subsequent output continues on the same line.

The program shown in Listing 2.1, called `Countdown`, makes use of both methods.

Study the output of `Countdown` carefully alongside the source code. Notice that the word "Liftoff" appears on the same line as the preceding countdown text, even though it is printed using `println`. This is because all the preceding `print` calls left the cursor on the same line, and `println` simply outputs its content at the current cursor position before advancing to the next line.

```java
// LISTING 2.1
//********************************************************************
// Countdown.java
//
// Demonstrates the difference between print and println.
//********************************************************************
public class Countdown
{
   //-----------------------------------------------------------------
   // Prints two lines of output representing a rocket countdown.
   //-----------------------------------------------------------------
   public static void main(String[] args)
   {
        System.out.print("Three...");
        System.out.print("Two...");
        System.out.print("One...");
        System.out.print("Zero...");

        System.out.println("Liftoff!"); // appears on the first output
line

        System.out.println("Houston, we have a problem.");
   }
}
```

O U T P U T

Three... Two... One... Zero... Liftoff!
Houston, we have a problem.

String Concatenation

A string literal cannot extend across multiple lines within a program. The following statement is syntactically invalid and will produce a compile-time error:

// The following statement will not compile

System.out.println ("The only stupid question is

the one that's not asked.");

When a string is too long to fit comfortably on a single line of code, we can use string concatenation to join two or more strings end to end. The string concatenation operator is the plus sign (+). The following expression joins two string literals into a single, longer string:

"The only stupid question is " + "the one that's not asked."

The `Facts` program in Listing 2.2 contains several `println` calls. The first one outputs a sentence that is too long to fit on one line of code. Since a string literal cannot span lines, the string is divided into two parts and joined using the concatenation operator. The result is a single combined string that is passed to `println` for output.

Also note that `println` can be called with no parameters at all, as shown in the second statement of the `Facts` program. When called this way, nothing visible is printed, but the cursor advances to the next line — effectively inserting a blank line in the output.

The final three `println` calls in `Facts` illustrate another important capability: strings can be concatenated with numeric values. The numbers in those statements are not enclosed in double quotes and are therefore not string literals — they are numeric values. In these cases, Java automatically converts the number to its string equivalent and then performs the concatenation.

Since we are working with fixed values here, we could just as easily have embedded the number directly inside the string literal, like so:

"Speed of ketchup: 40 km per year"

Numeric digits are perfectly valid characters within a string. The reason we keep them separate in the `Facts` program is to demonstrate the string-and-number concatenation capability explicitly, as it will prove useful in many upcoming examples.

KEY CONCEPT
In Java, the `+` operator is used both for addition and for string concatenation.

As you might anticipate, the `+` operator also serves as the arithmetic addition operator. What the `+` operator actually does in any given context depends entirely on the types of its operands. If one or both operands are strings, string concatenation is performed.

The `Addition` program in Listing 2.3 illustrates the contrast between these two behaviors.

In the first call to `println` in `Addition`, both `+` operations perform string concatenation. This is because operators are evaluated left to right: the first `+` joins the string with the number `24`, producing a longer string, and the second `+` then appends `45` to that string, producing an even longer one, which is ultimately printed.

In the second call to `println`, parentheses are used to group the two numeric operands together. This forces that operation to be evaluated first. Since both operands are numbers, arithmetic addition is performed, yielding `69`. That result is then concatenated with the preceding string and printed.

```java
// LISTING 2.2

//********************************************************************

// Facts.java

//

// Demonstrates the use of the string concatenation operator and the

// automatic conversion of an integer to a string.

//********************************************************************

public class Facts

{

   //-----------------------------------------------------------------

   // Prints various facts.

   //-----------------------------------------------------------------

   public static void main(String[] args)

   {

      // Strings can be concatenated into one long string

      System.out.println("We present the following facts for your "

                         + "extracurricular edification" );

      System.out.println();

      // A string can contain numeric digits

      System.out.println("Letters in the Hawaiian alphabet: 12");

      // A numeric value can be concatenated to a string

      System.out.println("Dialing code for Antarctica: " + 672);

      System.out.println("Year in which Leonardo da Vinci invented "

                        + "the parachute: " + 1515);

      System.out.println("Speed of ketchup: " + 40 + " km per year");

   }

}
```

O U T P U T

We present the following facts for your extracurricular edification:

Letters in the Hawaiian alphabet: 12

Dialing code for Antarctica: 672

Year in which Leonardo da Vinci invented the parachute: 1515

Speed of ketchup: 40 km per year

```
// LISTING 2.3

//*************************************************************************

// Addition.java

//

// Demonstrates the difference between the addition and string

// concatenation operators.

//*************************************************************************

public class Addition

{

   //-----------------------------------------------------------------

   // Concatenates and adds two numbers and prints the results.

   //-----------------------------------------------------------------

   public static void main(String[] args)

   {

       System.out.println("24 and 45 concatenated: " + 25 + 45);


       System.out.println("24 and 45 added: " + (24 + 45));

   }

}
```

O U T P U T

```
24 and 45 concatenated: 2445
24 and 45 added: 69
```

We will revisit this type of situation later in the chapter when we formalize the operator precedence rules that govern the order in which operations are carried out.

Escape Sequences

Because the double quotation character (`"`) is used in Java to mark the beginning and end of a string literal, printing a quotation character within a string requires a special technique. If we simply insert it directly into the string as in `""""`, the compiler interprets the second quotation mark as the end of the string and cannot make sense of the third, resulting in a compile-time error.

KEY CONCEPT
An escape sequence can be used to represent a character that would otherwise cause compilation problems.

To address this, Java defines a set of escape sequences for representing characters that cannot be included directly in a string. Each escape sequence begins with a backslash (`\`), which signals to the compiler that the character or characters that follow carry a special meaning rather than a literal one. Figure 2.1 lists all of the escape sequences defined in Java.

Escape Sequence	Meaning
`\b`	backspace
`\t`	tab
`\n`	newline
`\r`	carriage return
`\"`	double quote
`\'`	single quote
`\\`	backslash

FIGURE 2.1 Java escape sequences

The program in Listing 2.4, called `Roses`, prints text resembling a short poem using a single `println` statement, despite the poem spanning several lines of output. Pay close attention to the escape sequences scattered throughout the string. The `\n` sequence forces a line break, moving output to the beginning of the next line. The `\t` sequence inserts a tab character, producing indentation. (The exact width of the indentation may vary depending on the system's tab settings.) The `\"` sequence allows a literal double-quote character to appear within the string without terminating it.

```
// LISTING 2.4

//********************************************************************

// Roses.java

//

// Demonstrates the use of escape sequences.

//********************************************************************

public class Roses

{

   //-----------------------------------------------------------------

   // Prints a poem (of sorts) on multiple lines.

   //-----------------------------------------------------------------

   public static void main(String[] args)

   {

       System.out.println("Roses are red, \n\tViolets are blue,\n" +

                "Sugar is sweet, \n\tBut I have \"commitment
issues\",\n\t" +

                "So I'd rather just be friends\n\tAt this point in our "
+

                "relationship.");

   }

}
```

O U T P U T

Roses are red,

    Violets are blue,

Sugar is sweet,

    But I have "commitment issues",

    So I'd rather just be friends

    At this point in our relationship.

2.2 Variables and Assignment

The vast majority of the information a program works with is managed through variables. Let's examine how variables are declared and put to use.

Variables

A variable is a named location in memory where a data value can be stored. A variable declaration instructs the compiler to set aside a region of main memory large enough to hold a value of the specified type, and to associate that region with the name we provide.

KEY CONCEPT
A variable is a name for a memory location used to hold a value of a particular data type.

Consider the `PianoKeys` program shown in Listing 2.5. The first line of the `main` method declares a variable named `keys` that holds an integer (`int`) value, and initializes it to `88`. When no initial value is provided in a declaration, the variable's value is considered undefined. Most Java compilers will issue errors or warnings if you attempt to reference a variable before assigning it an explicit value.

The `keys` variable and its associated value can be visualized as follows:

keys | 88

Within `PianoKeys`, the call to `println` uses two pieces of information: a string literal and the variable `keys`. When a variable is referenced in an expression, the value it currently holds is retrieved and used. At the point of execution, `keys` holds the value `88`. Since that value is an integer, Java automatically converts it to a string and concatenates it with the preceding string literal before passing the combined result to `println` for output.

It is also possible to declare multiple variables of the same type in a single declaration statement. Each variable in the list may or may not be given an initial value:

int count, minimum = 0, result;

```
//LISTING 2.5

//*****************************************************************

// PianoKeys.java

//

// Demonstrates the declaration, initialization, and use of an

// integer variable.

//*****************************************************************

public class PianoKeys

{

   //-----------------------------------------------------------------

   // Prints the number of keys on a piano.

   //-----------------------------------------------------------------

   public static void main(String[] args)

   {

       int keys = 88;


       System.out.println("A piano has " + keys + " keys.");

   }

}
```

O U T P U T

A piano has 88 keys.

The Assignment Statement

Let's look at a program that modifies the value held by a variable. The `Geometry` program in Listing 2.6 begins by declaring an integer variable called `sides` and initializing it to `7`. It then prints the current value of `sides`.

The statement that follows in `main` is:

sides = 10;

This is an assignment statement — it stores a new value into an existing variable. When executed, the expression on the right-hand side of the assignment operator (`=`) is evaluated, and the resulting value is written into the memory location identified by the variable on the left-hand side. In this case the expression is simply the literal value `10`. More complex expressions will be discussed in the sections that follow.

KEY CONCEPT
Accessing data leaves it intact in memory, but an assignment statement overwrites the old data.

A variable can hold only one value at any given time. Assigning a new value to a variable replaces whatever was stored there previously. When `10` is assigned to `sides`, the original value of `7` is permanently overwritten:

After initialization: sides | 7

After first assignment: sides | 10

Reading the value of a variable — for example, printing it — does not alter what is stored there. This reflects the fundamental nature of computer memory: reading leaves data unchanged, while writing replaces existing data with new data.

KEY CONCEPT
We cannot assign a value of one type to a variable of an incompatible type.

Java is a strongly typed language, which means that assigning a value to a variable whose type is incompatible with that value is not permitted. Any such attempt will be caught and reported as an error during compilation. Consequently, the expression on the right-hand side of an assignment statement must evaluate to a value that is compatible with the declared type of the variable on the left-hand side.

```java
// LISTING 2.6

//********************************************************************

// Geometry.java

//

// Demonstrates the use of an assignment statement to change the

// value stored in a variable.

//********************************************************************

public class Geometry

{

   //-----------------------------------------------------------------

   // Prints the number of sides of several geometric shapes.

   //-----------------------------------------------------------------

   public static void main(String[] args)

   {

       int sides = 7; // declaration with initialization

       System.out.println("A heptagon has " + sides + " sides.");


       sides = 10;

       System.out.println("A decagon has " + sides + " sides.");


       sides = 12;

       System.out.println("A dodecagon has " + sides + " sides.");


   }

}
```

O U T P U T

A heptagon has 7 sides.

A decagon has 10 sides.

A dodecagon has 12 sides.

Constants

Some programs work with values that remain fixed from start to finish. Consider a program written for a theater that seats no more than 427 people. Rather than scattering the literal value `427` throughout the code, it is far better practice to assign it a meaningful name — such as `MAX_OCCUPANCY` — and use that name instead. Raw numeric literals embedded in code can leave readers wondering what they represent. A well-chosen name removes that ambiguity and makes the code immediately more readable.

Constants resemble variables in that they are identifiers associated with stored values. The critical difference is that, unlike variables, their value is fixed for the entire duration of their existence — they do not change.

In Java, placing the reserved word `final` before a declaration transforms the identifier into a constant. By convention, constant names are written entirely in uppercase letters, with individual words separated by underscores. This visual distinction makes them easy to tell apart from regular variables at a glance. The theater capacity constant, for example, would be declared as:

final int MAX_OCCUPANCY = 427;

If any part of the program attempts to assign a new value to a constant after its initial declaration, the compiler will flag it as an error. This behavior is one of the key advantages of using constants: they serve as a safeguard against accidental modifications that could introduce hard-to-detect bugs.

There is a third compelling reason to favor constants over embedded literals. If the value associated with a constant needs to change — say, the theater is renovated and its capacity grows from 427 to 535 — only the single declaration needs to be updated. Every reference to `MAX_OCCUPANCY` throughout the program automatically reflects the new value. If the literal `427` had been written directly into the code in multiple places, each occurrence would need to be tracked down and changed individually. Missing even one such occurrence would almost certainly lead to incorrect behavior.

2.3 Primitive Data Types

Java defines exactly eight primitive data types: four integer variants, two floating point variants, a character type, and a boolean type. Every other kind of data in Java is represented through objects. Let's take a closer look at each of these eight primitive types.

Integers and Floating Points

Java supports two fundamental categories of numeric values: integers, which carry no fractional component, and floating point numbers, which do. Within the integer category there are four distinct types — `byte`, `short`, `int`, and `long` — and within the floating point category there are two — `float` and `double`.

KEY CONCEPT
Java has two kinds of numeric values: integer and floating point. There are four integer data types and two floating point data types.

All six numeric types differ from one another in the amount of memory they consume, which in turn determines the range of values each can represent. Importantly, the size of each type is fixed and identical across all hardware platforms, which contributes to Java's portability. All numeric types are signed, meaning they are capable of holding both positive and negative values. Figure 2.2 provides a summary of all numeric primitive types along with their storage requirements and value ranges.

A bit — short for binary digit — can hold either a `0` or a `1`. Because each bit represents one of two possible states, a sequence of N bits is capable of representing 2^N distinct values. A more thorough treatment of number systems and binary representations can be found in Appendix B.

When designing programs, it is sometimes worth paying attention to the size of the variables we declare, particularly in memory-constrained environments such as programs running on personal data assistants (PDAs). In such cases, selecting a narrower type where a smaller range suffices can make a meaningful difference. For example, if a variable's value will always fall between 1 and 1000, a `short` (two bytes) is entirely sufficient. On the other hand, when the potential range of a value is uncertain, it is wise to be generous in our choice of type. In most everyday situations, memory capacity is not a significant concern and we can afford to lean toward larger types without worry.

It is also worth noting that while `float` can accommodate extremely large and extremely small values, it offers only seven significant digits of precision. If a calculation requires faithfully representing a value such as `50341.2077`, a `double` must be used instead.

As previously discussed, a literal is an explicit value written directly into the source code. The numeric values appearing in programs like `Facts`, `Addition`, and `PianoKeys` are all

integer literals. By default, Java treats any integer literal as type `int`. To designate a literal as type `long`, append an `L` or `l` to the end of the value — for example, `45L`.

Similarly, Java treats all floating point literals as `double` by default. To treat a floating point literal as a `float`, append an `F` or `f` — for example, `2.718F` or `123.45f`. A `double` literal may optionally be followed by a `D` or `d`, though this suffix is rarely necessary.

The following are representative examples of numeric variable declarations in Java:

int answer = 42;

byte smallNumber1, smallNumber2;

long countedStars = 86827263927L;

float ratio = 0.2363F;

double delta = 453.523311903;

Characters

Characters represent another foundational category of data that computers manage. Individual characters can be treated as standalone data items and, as we have already seen, can be combined to form character strings.

A character literal in Java is written using single quotation marks, such as `'b'`, `'J'`, or `';'`. Recall that string literals use double quotation marks, and that `String` is not a primitive type in Java — it is a class. We examine the `String` class in detail in the following chapter.

It is important to distinguish between a digit appearing as a character (or embedded in a string) and a digit used as a numeric value. The number `602` is a numeric quantity suitable for arithmetic. But in the string `"602 Greenbriar Court"`, the characters `6`, `0`, and `2` are simply characters, no different in nature from the letters and spaces around them.

The set of characters a language can work with is defined by its character set — an ordered listing of every character the language recognizes. Different programming languages have historically adopted different character sets. One longstanding standard is ASCII, which stands for the American Standard Code for Information Interchange. The original ASCII specification used 7 bits per character, accommodating 128 distinct characters, including:

- uppercase letters such as `'A'`, `'B'`, and `'C'`
- lowercase letters such as `'a'`, `'b'`, and `'c'`
- punctuation marks such as the period (`'.'`), semicolon (`';'`), and comma (`','`)
- the digit characters `'0'` through `'9'`
- the space character `' '`

- special symbols such as the ampersand (`'&'`), vertical bar (`'|'`), and backslash (`'\'`)
- control characters such as carriage return, null, and end-of-text markers

Control characters are sometimes described as nonprinting or invisible because they have no visible symbol associated with them. Nonetheless, they are fully valid characters that can be stored and manipulated like any other. Many of them carry special significance for particular software applications.

As computing expanded into a global endeavor, demand grew for character sets capable of representing the alphabets of many different languages. ASCII was extended to 8 bits, doubling its capacity to 256 characters and adding accented and diacritical characters used in languages other than English.

KEY CONCEPT
Java uses the 16-bit Unicode character set to represent character data.

Even 256 characters proved insufficient for the world's writing systems, especially those of Asian languages with their extensive inventories of ideographic characters. The designers of Java therefore adopted the Unicode character set, which allocates 16 bits per character, enabling the representation of 65,536 unique characters. Unicode encompasses characters and symbols from a wide range of the world's languages. ASCII itself is a proper subset of Unicode. A more comprehensive discussion of the Unicode character set appears in Appendix C.

Because every character in a character set is assigned a specific numeric code, characters exist in a well-defined sequence known as lexicographic order. In both ASCII and Unicode, the digit characters `'0'` through `'9'` are assigned consecutive codes with no gaps between them, as are the lowercase letters `'a'` through `'z'` and the uppercase letters `'A'` through `'Z'`. This ordered, contiguous arrangement makes operations such as alphabetical sorting straightforward to implement. Sorting algorithms are covered in Chapter 13.

In Java, the `char` data type holds a single character. The following are examples of character variable declarations:

```
char topGrade = 'A';

char symbol1, symbol2, symbol3;

char terminator = ';', separator = ' ';
```

Booleans

Here is the reconstructed text:

2.3 Primitive Data Types

Java defines exactly eight primitive data types: four integer variants, two floating point variants, a character type, and a boolean type. Every other kind of data in Java is represented through objects. Let's take a closer look at each of these eight primitive types.

Integers and Floating Points

Java supports two fundamental categories of numeric values: integers, which carry no fractional component, and floating point numbers, which do. Within the integer category there are four distinct types — `byte`, `short`, `int`, and `long` — and within the floating point category there are two — `float` and `double`.

KEY CONCEPT
Java has two kinds of numeric values: integer and floating point. There are four integer data types and two floating point data types.

All six numeric types differ from one another in the amount of memory they consume, which in turn determines the range of values each can represent. Importantly, the size of each type is fixed and identical across all hardware platforms, which contributes to Java's portability. All numeric types are signed, meaning they are capable of holding both positive and negative values. Figure 2.2 provides a summary of all numeric primitive types along with their storage requirements and value ranges.

A bit — short for binary digit — can hold either a `0` or a `1`. Because each bit represents one of two possible states, a sequence of N bits is capable of representing 2^N distinct values. A more thorough treatment of number systems and binary representations can be found in Appendix B.

When designing programs, it is sometimes worth paying attention to the size of the variables we declare, particularly in memory-constrained environments such as programs running on personal data assistants (PDAs). In such cases, selecting a narrower type where a smaller range suffices can make a meaningful difference. For example, if a variable's value will always fall between 1 and 1000, a `short` (two bytes) is entirely sufficient. On the other hand, when the potential range of a value is uncertain, it is wise to be generous in our choice of type. In most everyday situations, memory capacity is not a significant concern and we can afford to lean toward larger types without worry.

It is also worth noting that while `float` can accommodate extremely large and extremely small values, it offers only seven significant digits of precision. If a calculation requires faithfully representing a value such as `50341.2077`, a `double` must be used instead.

As previously discussed, a literal is an explicit value written directly into the source code. The numeric values appearing in programs like `Facts`, `Addition`, and `PianoKeys` are all integer literals. By default, Java treats any integer literal as type `int`. To designate a literal as type `long`, append an `L` or `l` to the end of the value — for example, `45L`.

Similarly, Java treats all floating point literals as `double` by default. To treat a floating point literal as a `float`, append an `F` or `f` — for example, `2.718F` or `123.45f`. A `double` literal may optionally be followed by a `D` or `d`, though this suffix is rarely necessary.

The following are representative examples of numeric variable declarations in Java:

java

```
int answer = 42;

byte smallNumber1, smallNumber2;

long countedStars = 86827263927L;

float ratio = 0.2363F;

double delta = 453.523311903;
```

Characters

Characters represent another foundational category of data that computers manage. Individual characters can be treated as standalone data items and, as we have already seen, can be combined to form character strings.

A character literal in Java is written using single quotation marks, such as `'b'`, `'J'`, or `';'`. Recall that string literals use double quotation marks, and that `String` is not a primitive type in Java — it is a class. We examine the `String` class in detail in the following chapter.

It is important to distinguish between a digit appearing as a character (or embedded in a string) and a digit used as a numeric value. The number `602` is a numeric quantity suitable for arithmetic. But in the string `"602 Greenbriar Court"`, the characters `6`, `0`, and `2` are simply characters, no different in nature from the letters and spaces around them.

The set of characters a language can work with is defined by its character set — an ordered listing of every character the language recognizes. Different programming languages have historically adopted different character sets. One longstanding standard is ASCII, which stands for the American Standard Code for Information Interchange. The original ASCII specification used 7 bits per character, accommodating 128 distinct characters, including:

- uppercase letters such as `'A'`, `'B'`, and `'C'`
- lowercase letters such as `'a'`, `'b'`, and `'c'`
- punctuation marks such as the period (`'.'`), semicolon (`';'`), and comma (`','`)
- the digit characters `'0'` through `'9'`
- the space character `' '`
- special symbols such as the ampersand (`'&'`), vertical bar (`'|'`), and backslash (`'\'`)
- control characters such as carriage return, null, and end-of-text markers

Control characters are sometimes described as nonprinting or invisible because they have no visible symbol associated with them. Nonetheless, they are fully valid characters that can be stored and manipulated like any other. Many of them carry special significance for particular software applications.

As computing expanded into a global endeavor, demand grew for character sets capable of representing the alphabets of many different languages. ASCII was extended to 8 bits, doubling its capacity to 256 characters and adding accented and diacritical characters used in languages other than English.

KEY CONCEPT
Java uses the 16-bit Unicode character set to represent character data.

Even 256 characters proved insufficient for the world's writing systems, especially those of Asian languages with their extensive inventories of ideographic characters. The designers of Java therefore adopted the Unicode character set, which allocates 16 bits per character, enabling the representation of 65,536 unique characters. Unicode encompasses characters and symbols from a wide range of the world's languages. ASCII itself is a proper subset of Unicode. A more comprehensive discussion of the Unicode character set appears in Appendix C.

Because every character in a character set is assigned a specific numeric code, characters exist in a well-defined sequence known as lexicographic order. In both ASCII and Unicode, the digit characters `'0'` through `'9'` are assigned consecutive codes with no gaps between them, as are the lowercase letters `'a'` through `'z'` and the uppercase letters `'A'` through `'Z'`. This ordered, contiguous arrangement makes operations such as alphabetical sorting straightforward to implement. Sorting algorithms are covered in Chapter 13.

In Java, the `char` data type holds a single character. The following are examples of character variable declarations:

java

```
char topGrade = 'A';

char symbol1, symbol2, symbol3;

char terminator = ';', separator = ' ';
```

Booleans

A boolean value, declared in Java using the reserved word `boolean`, is restricted to exactly two valid states: `true` and `false`. Boolean variables are most commonly used to record whether a particular condition holds, but they are equally suited to any situation involving a binary distinction — such as whether a light is on or off. The name "boolean" honors the English mathematician George Boole, who in the nineteenth century developed the algebraic system bearing his name, in which variables take only one of two possible values.

A `boolean` cannot be converted to any other primitive type, and no other primitive type can be converted to a `boolean`. The words `true` and `false` are reserved literals in Java and cannot be used for any other purpose.

The following are examples of boolean variable declarations:

boolean flag = true;

boolean tooHigh, tooSmall, tooRough;

boolean done = false;

2.4 Expressions

An expression is a construct consisting of one or more operators and their associated operands that together compute a value. The result of that computation need not be numeric, though it very often is. The operands in an expression may be literals, constants, variables, or other data sources. Understanding how expressions are formed and evaluated is central to programming. In this section we focus specifically on arithmetic expressions — those involving numeric operands and yielding numeric results.

KEY CONCEPT

Expressions are combinations of operators and operands used to perform a calculation.

Arithmetic Operators

The standard arithmetic operations — addition (`+`), subtraction (`−`), multiplication (`*`), and division (`/`) — are defined for all numeric types in Java, both integer and floating point. Java also provides a fifth arithmetic operator: the remainder operator (`%`), which returns what is left over after dividing the first operand by the second. This operator is also known as the modulus operator. The sign of the result matches the sign of the numerator (the left-hand operand). A few examples are shown below:

Operation	Result
`17 % 4`	`1`
`-20 % 3`	`-2`

```
10 % -5   0

3 % 8     3
```

When either or both operands of a numeric operator are floating point values, the result is a floating point value. However, the behavior of the division operator (`/`) deserves special attention.

KEY CONCEPT
The type of result produced by arithmetic division depends on the types of the operands.

When both operands are integers, `/` performs integer division, meaning any fractional portion of the result is discarded entirely. When at least one operand is a floating point value, `/` performs floating point division and retains the fractional part. To illustrate: `10 / 4` evaluates to `2`, whereas `10.0 / 4`, `10 / 4.0`, and `10.0 / 4.0` all evaluate to `2.5`.

Operators can also be classified by the number of operands they require. A unary operator acts on a single operand, while a binary operator operates on two. Both `+` and `−` can serve as either unary or binary operators. In their binary form they perform addition and subtraction; in their unary form they express positive and negative values. For example, `-1` uses the unary negation operator. The unary `+` operator exists but is rarely applied in practice.

Java does not include a built-in operator for exponentiation. However, the `Math` class supplies methods for this and many other mathematical operations, which we explore in Chapter 3.

Operator Precedence

Operators can be combined to form more complex expressions. Consider the following:

result = 14 + 8 / 2;

The entire right-hand side is evaluated before the result is stored in `result`. But what does it evaluate to? If addition were performed first, the result would be `11`; if division came first, it would be `18`. The order of evaluation matters enormously. In this case, division takes precedence over addition, so the result is `18`.

KEY CONCEPT
Java follows a well-defined set of precedence rules that governs the order in which operators will be evaluated in an expression.

All expressions are governed by an operator precedence hierarchy — a formal set of rules that determines the order in which operations are carried out. Java's arithmetic precedence rules parallel those familiar from algebra. Multiplication, division, and remainder share equal precedence and are all evaluated before addition and subtraction, which also share equal precedence with each other.

When two operators share the same precedence level, they are evaluated from left to right — what is called left-to-right associativity.

Parentheses override the default precedence order. If we wanted addition to be performed first in the previous example, we would write:

```
result = (14 + 8) / 2;
```

Any sub-expression enclosed in parentheses is evaluated before anything outside it. In complex expressions, using parentheses even when technically unnecessary is good practice: it makes the intended evaluation order explicit and reduces the chance of misreading the code.

Parentheses can be nested, with the innermost expressions evaluated first:

```
result = 3 * ((18 – 4) / 2);
```

Here, the result is 21. The subtraction is forced first by the inner parentheses. Then, even though multiplication and division normally share the same precedence and would be evaluated left to right, the outer parentheses force the division to occur before the multiplication. The multiplication concludes last.

Once all arithmetic is complete, the computed value is stored in the variable on the left-hand side of the assignment operator (=). This means the assignment operator has lower precedence than all arithmetic operators.

The evaluation order of an expression can also be visualized using an expression tree, as shown in Figure 2.3. In such a tree, operators lower in the structure are evaluated before those higher up, either because of their inherent precedence or because they are enclosed within parentheses. The parentheses themselves function as operators and carry higher precedence than nearly everything else. Figure 2.4 presents a precedence table covering arithmetic operators, parentheses, and the assignment operator. A complete precedence table encompassing all Java operators can be found in Appendix D.

For an expression to be syntactically valid, every opening parenthesis must have a corresponding closing parenthesis, and they must be properly nested. The following are examples of invalid expressions:

```
result = ((19 + 8) % 3) – 4);    // not valid

result = (19 (+ 8 %) 3 – 4);     // not valid
```

When a variable appears in an expression, its current value is retrieved and used in the computation. In the following statement:

sum = count + total;

the current values of `count` and `total` are added together and the result is stored in `sum`. The previous value of `sum` is overwritten, while the values of `count` and `total` remain unchanged.

A variable may appear on both sides of an assignment operator simultaneously. For example, if `count` currently holds the value `15`:

count = count + 1;

The right-hand expression is evaluated first: the current value of `count` (which is `15`) is retrieved, `1` is added to it, yielding `16`, and that result is then stored back into `count`. The original value of `15` is replaced by `16`. This is the standard idiom for incrementing a variable by one.

As a further illustration of expression evaluation, the `TempConverter` program in Listing 2.7 converts a Celsius temperature to its Fahrenheit equivalent using the formula:

Fahrenheit = (9/5) × Celsius + 32

In the program, the division operands are written as floating point literals to ensure the fractional component is preserved. The precedence rules ensure that multiplication is carried out before addition in the final computation.

As currently written, `TempConverter` is of limited utility — it converts only the single hardcoded value of 24 degrees Celsius and produces the same output every time it runs. A genuinely useful version would accept the input value from the user at runtime. Interactive input is covered later in this chapter.

```
// LISTING 2.7

//********************************************************************

// TempConverter.java Java Foundations

//

// Demonstrates the use of primitive data types and arithmetic

// expressions.

//********************************************************************

public class TempConverter
```

```
{
   //-----------------------------------------------------------------
   // Computes the Fahrenheit equivalent of a specific Celsius
   // value using the formula F = (9/5)C + 32.
   //-----------------------------------------------------------------
   public static void main(String[] args)
   {
       final int BASE = 32;
       final double CONVERSION_FACTOR = 9.0 / 5.0;

       double fahrenheitTemp;
       int celsiusTemp = 24; // value to convert

       fahrenheitTemp = celsiusTemp * CONVERSION_FACTOR + BASE;

       System.out.println("Celsius Temperature: " + celsiusTemp);
       System.out.println("Fahrenheit Equivalent: " + fahrenheitTemp);
   }
}
```

O U T P U T

Celsius Temperature: 24

Fahrenheit Equivalent: 75.2

Increment and Decrement Operators

Java provides two additional shorthand arithmetic operators. The increment operator (++) adds 1 to any integer or floating point variable. The two plus signs must appear immediately adjacent to each other — no white space is permitted between them. The decrement

operator (`--`) works symmetrically, subtracting `1` from the variable. Both are unary operators, acting on a single operand. The following statement increments `count` by one:

count++;

The updated value is written back into `count`, making this statement functionally equivalent to:

count = count + 1;

Both the increment and decrement operators can be placed either after the variable (postfix form: `count++`, `count--`) or before it (prefix form: `++count`, `--count`). When either form appears as a standalone statement, the two are interchangeable — it makes no difference whether you write `count++` or `++count`. That said, when used as a standalone statement, the postfix form is the conventional choice.

The distinction between prefix and postfix becomes significant when the operator appears within a larger expression. If `count` currently holds `15`:

total = count++;

This assigns the value `15` to `total` (the original value of `count`) and then increments `count` to `16`. In contrast:

total = ++count;

This increments `count` to `16` first and then assigns that new value to `total`, so both `total` and `count` end up holding `16`.

In both cases `count` is incremented, but the value contributed to the surrounding expression depends entirely on whether the prefix or postfix form is used. Because of these subtle differences, these operators should be used with deliberate care. When in doubt, favor the form that makes the code most readable.

Assignment Operators

Java offers a set of compound assignment operators that combine an arithmetic operation with assignment in a single, concise notation. For example, the `+=` operator:

total += 5;

is equivalent to:

total = total + 5;

The right-hand side of a compound assignment can be a full expression. That expression is evaluated in its entirety first, and the result is then combined with the current value of the left-hand variable using the appropriate operation. For instance:

total += (sum - 12) / count;

is equivalent to:

total = total + ((sum - 12) / count);

Java defines analogous compound assignment operators for the other arithmetic operations: `-=` for subtraction, `*=` for multiplication, `/=` for division, and `%=` for remainder. A complete listing of all Java operators appears in Appendix D.

In every compound assignment operator, the entire right-hand side expression is fully evaluated before being combined with the left-hand variable. Therefore:

result *= count1 + count2;

is equivalent to:

result = result * (count1 + count2);

And likewise:

result %= (highest - 40) / 2;

is equivalent to:

result = result % ((highest - 40) / 2);

Just as with their non-compound counterparts, some compound assignment operators adapt their behavior based on the types of the operands involved. For example, when `+=` is applied to string operands, it performs string concatenation rather than numeric addition.

2.5 Data Conversion

Because Java is a strongly typed language, every value in a program is bound to a specific type. There are situations where it becomes necessary or convenient to treat a value as though it belongs to a different type — but such conversions must be approached with care, since information can be lost in the process. For instance, if a `short` variable holding the value `1000` is converted to a `byte`, the `byte` does not have sufficient bits to represent that value, and the stored result will bear no meaningful relationship to the original number.

Conversions between primitive types fall into two broad categories: widening conversions and narrowing conversions. Widening conversions are generally safe because they move a

value into a type with equal or greater storage capacity, preserving the original value without loss. Figure 2.5 enumerates the widening conversions Java supports.

As an example, converting a `byte` to a `short` is a widening conversion: a `byte` uses 8 bits while a `short` uses 16, so no information is discarded. Conversions that stay within the integer family or within the floating point family preserve the exact numeric value.

One nuance worth noting: widening conversions that cross from an integer type to a floating point type can lose precision, even if no magnitude is lost. Converting an `int` or `long` to a `float`, or a `long` to a `double`, may cause some of the least significant digits to be dropped. The result will be a rounded approximation of the original value, computed according to the rounding rules defined by the IEEE 754 floating point standard.

KEY CONCEPT
Narrowing conversions should be avoided because they can lose information.

Narrowing conversions carry greater risk. They typically move a value into a type with smaller storage capacity, which can result in the loss of both magnitude and precision. Figure 2.6 lists the narrowing conversions available in Java.

One exception to the "smaller storage" characterization of narrowing conversions involves converting a `byte` (8 bits) or `short` (16 bits) to a `char` (also 16 bits). Despite the equal size, these are still classified as narrowing conversions because the sign bit is reinterpreted. Since `char` is an unsigned type, a negative integer value converted to a `char` will yield a character whose numeric code bears no predictable relationship to the original value.

It is also worth noting that `boolean` values are entirely isolated from the conversion system. A `boolean` cannot be widened or narrowed to any other primitive type, and no other primitive type can be converted to `boolean`.

Conversion Techniques

Java provides three distinct mechanisms for performing type conversions:

- assignment conversion
- promotion
- casting

Assignment conversion takes place automatically when a value of one type is assigned to a variable of a different type. Only widening conversions are handled this way. For example, if `money` is a `float` variable and `dollars` is an `int` variable, the following assignment silently converts the integer to a float:

```
money = dollars;
```

If `dollars` holds `25`, then `money` will contain `25.0` after the assignment. The reverse — assigning `money` to `dollars` — would be rejected by the compiler, which recognizes it as a potentially lossy narrowing conversion. To force such an assignment, an explicit cast is required.

Promotion is a form of implicit conversion that occurs automatically when an operator needs its operands to be of a particular type to carry out the operation. For example, when a floating point variable `sum` is divided by an integer variable `count`, Java promotes `count` to a floating point value before performing the division, producing a floating point result:

result = sum / count;

A similar promotion occurs during string concatenation: when a number is joined to a string using `+`, the number is silently converted to its string representation before the concatenation takes place.

Casting is the most explicit and comprehensive conversion mechanism Java provides. Any conversion that is possible in Java at all can be expressed using a cast. A cast is written as a type name enclosed in parentheses, placed immediately before the value or variable to be converted:

dollars = (int) money;

The cast extracts the value stored in `money` and truncates any fractional part, returning the integer portion only. If `money` contains `84.69`, then `dollars` will receive the value `84`. Crucially, the cast does not modify `money` itself — after the assignment, `money` still holds `84.69`.

Casts are particularly useful when we need to temporarily treat a value as a different type. For example, to divide one integer by another and obtain a floating point result:

result = (float) total / count;

The cast converts `total` to a `float` without altering its stored value. Java then promotes `count` to a `float` via arithmetic promotion, and floating point division is performed, yielding the intended fractional result. Without the cast, both operands would have been treated as integers, the division would have discarded the fractional part, and `result` would have received a truncated integer value converted to a float. Note also that because the cast operator has higher precedence than the division operator, the cast applies to the value of `total` alone — not to the result of the division.

2.6 Reading Input Data

Designing a program to accept data from the user at runtime, rather than having values fixed in the source code, makes it far more versatile. Each time such a program runs, it can work with different input and produce correspondingly different results.

The Scanner Class

The `Scanner` class, part of Java's standard class library, offers a convenient and flexible set of methods for reading input values of many different types. Input can originate from several sources — text typed interactively by the user, data stored in a file, or even a character string that needs to be broken into components. Figure 2.7 summarizes the most commonly used methods provided by the `Scanner` class.

KEY CONCEPT
The `Scanner` class provides methods for reading input of various types from various sources.

Before any of its methods can be called, a `Scanner` object must be created. Objects in Java are instantiated using the `new` operator. The following declaration creates a `Scanner` configured to read from the keyboard:

```
Scanner scan = new Scanner(System.in);
```

This statement declares a variable named `scan` that holds a reference to a newly created `Scanner` object. The object is brought into existence by the `new` operator, which calls a special initialization method known as a constructor. The constructor receives a parameter that identifies the input source. `System.in` is the standard input stream, which by default corresponds to the keyboard. Object creation with `new` is explored further in the next chapter.

Unless configured otherwise, a `Scanner` uses whitespace characters — spaces, tabs, and newlines — as delimiters that separate individual input tokens from one another. The set of delimiters can be customized if the input uses a different separator character.

The `next` method reads the next whitespace-delimited token from the input and returns it as a string. If the input consists of a series of words separated by spaces, successive calls to `next` will return each word in turn. The `nextLine` method, by contrast, reads everything remaining on the current input line and returns it as a single string.

The `Echo` program in Listing 2.8 demonstrates basic use of the `Scanner`. It reads a line of text entered by the user, stores it in a `String` variable, and then prints it back to the screen. In the output shown below the listing, user-supplied input is rendered in red.

The `import` declaration at the top of the `Echo` class notifies the compiler that this program uses the `Scanner` class, which belongs to the `java.util` library. The role of `import` declarations is discussed more thoroughly in Chapter 3.

In addition to `nextLine`, the `Scanner` class provides dedicated methods for reading values of specific primitive types, such as `nextInt` and `nextDouble`. The `GasMileage` program in Listing 2.9 reads a trip's distance as an integer and the fuel consumed as a `double`, then computes and displays the miles-per-gallon figure.

As the output of `GasMileage` illustrates, the result is a floating point value with many decimal places. In the next chapter we introduce classes that help format output in more controlled ways, including rounding floating point results to a specified number of decimal places.

A `Scanner` processes its input one token at a time, according to whichever reading method is invoked and whichever delimiter pattern is currently in use. Multiple input values can therefore appear on the same line or be spread across several lines, depending on what is most natural for the situation.

Chapter 5 revisits the `Scanner` class in the context of reading from data files and working with custom delimiters. Appendix H provides a deeper look at using `Scanner` with regular expressions for more sophisticated input parsing.

```
// LISTING 2.8

//********************************************************************

// Echo.java Java Foundations

//

// Demonstrates the use of the nextLine method of the Scanner class

// to read a string from the user.

//********************************************************************


import java.util.Scanner;


public class Echo

{

   //-----------------------------------------------------------------
```

```
   // Reads a character string from the user and prints it.
   //-----------------------------------------------------------------
   public static void main(String[] args)
   {
       String message;
       Scanner scan = new Scanner(System.in);

       System.out.println("Enter a line of text:");
       message = scan.nextLine();

       System.out.println("You entered: \"" + message + "\"");
   }
}
```

OUTPUT

Enter a line of text:

Set your laser printer on stun!

You entered: "Set your laser printer on stun!"

```
// LISTING 2.9
//********************************************************************
// GasMileage.java Java Foundations
//
// Demonstrates the use of the Scanner class to read numeric data.
//********************************************************************

import java.util.Scanner;
```

```java
public class GasMileage

{

   //-----------------------------------------------------------------

   // Calculates fuel efficiency based on values entered by the

   // user.

   //-----------------------------------------------------------------

   public static void main(String[] args)

   {

       int miles;

       double gallons, mpg;


       Scanner scan = new Scanner(System.in);


       System.out.println("Enter the number of miles: ");

       miles = scan.nextInt();


       System.out.println("Enter the gallons of fuel used: ");

       gallons = scan.nextDouble();


       mpg = miles / gallons;


       System.out.println("Miles Per Gallon: " + mpg);

   }

}
```

O U T P U T

Enter the number of miles: 369

Enter the gallons of fuel used: 12.4

Miles Per Gallon: 29.758064516129032

Summary of Key Concepts

■ The `print` and `println` methods are two distinct output services made available through the `System.out` object.

■ In Java, the `+` operator serves a dual purpose: it performs arithmetic addition when applied to numeric operands, and string concatenation when one or both operands are strings.

■ Escape sequences provide a way to include characters in a string that would otherwise interfere with compilation or be impossible to represent directly.

■ A variable is an identifier bound to a specific location in memory, used to store a value of a declared data type.

■ Reading the value of a variable leaves the contents of memory undisturbed, whereas an assignment statement replaces whatever value was previously stored there.

■ Java's strong typing rules prohibit assigning a value to a variable whose declared type is incompatible with that value.

■ Constants are named identifiers whose assigned value remains fixed and unchanging throughout the entire lifetime of their existence.

■ Java supports two fundamental categories of numeric data: integers and floating point numbers. The integer category comprises four distinct types, and the floating point category comprises two.

■ Java represents character data using the Unicode character set, which encodes each character in 16 bits and supports 65,536 unique characters.

■ An expression is a construct built from one or more operators and their operands, evaluated to produce a computed result.

■ The outcome of an arithmetic division operation is determined by the types of its operands — integer operands yield an integer result, while the presence of a floating point operand produces a floating point result.

■ Java defines a strict and well-specified operator precedence hierarchy that dictates the order in which the operators within an expression are evaluated.

■ Narrowing conversions — those that move a value into a type with reduced storage capacity — carry the risk of losing information and should generally be avoided.

■ The `Scanner` class supplies a versatile set of methods capable of reading input values of multiple data types from a variety of input sources.